Plan séquentiel multi-fidélité pour le co-krigeage

Méthode de Le Gratiet & Cannamela (2015) corrigée — note d'implémentation

`Multifidelity-GP-Clean/vpp_acceleration` · s'appuie sur `mfego/src/cokriging.py` (lecture seule) · 19/10/2026

1 Objectif et principe

On dispose de L codes de calcul hiérarchisés (niveaux $l = 1, \dots, L$, coûts $c_1 < \cdots < c_L$) et d'un co-krigeage `CoKriging` ajusté sur un plan *imbriqué* $\mathcal{D}_L \subset \cdots \subset \mathcal{D}_1$. On veut un modèle du niveau L **précis sur tout le domaine** $\mathcal{Q}$ — ce n'est pas la recherche d'un optimum, rôle de l'EGO de mfego. À chaque itération, il faut deux décisions : *où* calculer ($\boldsymbol{x}_0$) et *jusqu'à quel niveau* (l). Calculer $\boldsymbol{x}_0$ au niveau l impose de le calculer aux niveaux $1, \dots, l$: le plan reste imbriqué, condition exigée par `CoKriging.fit`.

Le Gratiet & Cannamela [1] choisissent $\boldsymbol{x}_0$ au maximum d'une variance de prédiction « ajustée » par validation croisée, puis l par une approximation de la réduction de l'erreur intégrée (IMSE) pondérée par le rapport des coûts. Nous gardons cette architecture, avec trois changements :

1. le choix du niveau compare des réductions de variance **mesurées sur la sortie haute fidélité** et calculées **exactement** (§ 4). La règle publiée omet un facteur ρ^2 et utilise un volume non borné ;
2. l'ajustement par validation croisée se réduit au leave-one-out (LOO) *de chaque GP de biais seul* (§ 3) : les éq. 6–7 de [1] deviennent inutiles ;
3. la cellule de Voronoï est définie dans la métrique du noyau (invariante par changement d'unités).

2 Modèle et décomposition de la variance

Le modèle auto-régressif de Kennedy & O'Hagan [2, 3, 4] s'écrit

$$Z_1 = \delta_1, \qquad Z_l(\boldsymbol{x}) = \rho_{l-1}\, Z_{l-1}(\boldsymbol{x}) + \delta_l(\boldsymbol{x}), \qquad \delta_1, \dots, \delta_L \text{ GP indépendants}, \qquad z_l = Z_l(\mathcal{D}_l) + \varepsilon_l,\ \varepsilon_l \sim \mathcal{N}(0, \tau_l^2 \boldsymbol{I}). \tag{1}$$

On note $P_j = \prod_{k=j}^{L-1} \rho_k$ ($P_L = 1$), de sorte que $Z_L = \sum_j P_j\, \delta_j$: P_j est le « gain » avec lequel la correction apportée par le niveau j se propage jusqu'à la sortie haute fidélité. On note $\boldsymbol{d}_j = z_j - \rho_{j-1} z_{j-1}(\mathcal{D}_j)$ les résidus observés ($\boldsymbol{d}_1 = z_1$), calculables grâce à l'imbrication, et μ_j, $k_j^n(\boldsymbol{x}, \boldsymbol{x}')$, $\sigma_j^2(\boldsymbol{x}) = k_j^n(\boldsymbol{x}, \boldsymbol{x})$ la moyenne, la covariance et la variance a posteriori de δ_j sachant $\boldsymbol{d}_j$ seul.

Proposition 1 (décomposition, [3] prop. 3.1, [1] éq. 4)**.** *Si le plan est imbriqué et les niveaux $j < L$ sans bruit, alors, à hyperparamètres fixés,*

$$\mathbb{E}[Z_L(\boldsymbol{x}) \mid z] = \sum_{j=1}^{L} P_j\, \mu_j(\boldsymbol{x}), \qquad \mathrm{Var}[Z_L(\boldsymbol{x}) \mid z] = \sum_{j=1}^{L} P_j^2\, \sigma_j^2(\boldsymbol{x}). \tag{2}$$

Preuve. L'application $(z_1, \dots, z_L) \mapsto (\boldsymbol{d}_1, \dots, \boldsymbol{d}_L)$ est linéaire et inversible (triangulaire par blocs, ce qui demande z_{j-1} observé sur $\mathcal{D}_j$, donc l'imbrication). Or $\boldsymbol{d}_j$ ne dépend que de δ_j. Les δ_j étant indépendants, conditionner par z revient à conditionner chaque δ_j par son propre $\boldsymbol{d}_j$. □

Lecture physique. $\sigma_j^2(\boldsymbol{x})$ est ce qu'on ignore encore de la correction apportée par le niveau j ; P_j^2 dit combien cette ignorance pèse sur la sortie haute fidélité. C'est cette décomposition qui permet de décider *quel* code lancer.

Dans le code

- δ_j est le GP `model.gps[j-1]`. `CoKriging.fit` l'ajuste sur les entrées ramenées au cube unité, $\boldsymbol{u}$ = `model._unit(x)`, et sur le résidu $\boldsymbol{d}_j$. Ses longueurs de corrélation $\theta_{j,m}$ (`gp.kernel.lengthscale`) sont donc sans dimension.
- P_j s'obtient par `model._factors(np.array([L]))[0, j-1]`. Les ρ sont dans `model.rhos` (`rhos[l-2]` relie $l-1$ à l) ; ce sont des estimations ponctuelles (MAP, prior lognormale en v3).
- $\sigma_j^2(\boldsymbol{u})$ vaut `gp.predict_batch(u)[1] - gp.get_noise_variance()`, et τ_j^2 vaut `gp.get_noise_variance()`.
- **Vérification** (cas 2D à 3 niveaux, plan 40/15/6) : $\sum_j P_j^2 \sigma_j^2$ égale la variance de `model.predict_batch(x, include_noise=False)` à $1{,}2 \cdot 10^{-4}$ près en relatif. L'écart vient du bruit τ_j^2 appris par mfego : le résidu $\boldsymbol{d}_j$ contient $\rho^2 \tau_{j-1}^2$, si bien que l'égalité (2) n'est exacte qu'à l'ordre $\rho^2 \tau_{j-1}^2 / \sigma^2$.

3 Où calculer : variance ajustée par validation croisée

Pourquoi la variance seule ne suffit pas. La variance d'un GP, $\sigma^2(\boldsymbol{x}) = k(\boldsymbol{x}, \boldsymbol{x}) - \boldsymbol{k}(\boldsymbol{x})^{\mathsf{T}} \widetilde{\boldsymbol{K}}^{-1} \boldsymbol{k}(\boldsymbol{x})$ avec $\widetilde{\boldsymbol{K}} = \boldsymbol{K} + \tau^2 \boldsymbol{I}$, *ne dépend pas des valeurs observées*. Seules la géométrie du plan et les hyperparamètres y entrent.

Or ces hyperparamètres sont uniques sur tout $\mathcal{Q}$ (noyau stationnaire) : le modèle suppose une « rugosité » uniforme. Pensons à un phénomène localisé, comme un décrochage ou une ventilation : là où la fonction varie plus vite que prévu, l'erreur réelle dépasse σ, et la variance ne le voit pas. Le LOO, lui, mesure des erreurs réellement observées [1, 13].

LOO en forme close. Pour un GP de moyenne m, avec $\boldsymbol{\alpha} = \widetilde{\boldsymbol{K}}^{-1}(\boldsymbol{y} - m)$ et des hyperparamètres fixés, l'erreur et la variance de prédiction de y_i sans le point i valent [5], [6, éq. 5.12] :

$$e_i = \frac{\alpha_i}{[\widetilde{\boldsymbol{K}}^{-1}]_{ii}}, \qquad s_i^2 = \frac{1}{[\widetilde{\boldsymbol{K}}^{-1}]_{ii}}, \qquad z_i^2 = \frac{e_i^2}{s_i^2} = \frac{\alpha_i^2}{[\widetilde{\boldsymbol{K}}^{-1}]_{ii}}. \tag{3}$$

Si le modèle est correct, $z_i \sim \mathcal{N}(0,1)$: $z_i^2 \gg 1$ signale une erreur sous-estimée. z_i^2 est invariant par changement d'échelle de y, on peut donc utiliser directement `gp.alpha` (sorties normalisées).

Proposition 2 (LOO par niveau)**.** *Sous les hypothèses de la proposition 1, soit $\boldsymbol{x}_i \in \mathcal{D}_j$, et $e_j(\boldsymbol{x}_i)$, $s_j^2(\boldsymbol{x}_i)$ l'erreur et la variance LOO du co-krigeage au niveau j quand $\boldsymbol{x}_i$ est retiré de tous les niveaux. Alors*

$$e_j(\boldsymbol{x}_i) - \rho_{j-1}\, e_{j-1}(\boldsymbol{x}_i) = e_i^{(\delta_j)}, \qquad s_j^2(\boldsymbol{x}_i) - \rho_{j-1}^2\, s_{j-1}^2(\boldsymbol{x}_i) = s_i^{2\,(\delta_j)},$$

où $e_i^{(\delta_j)}$, $s_i^{2\,(\delta_j)}$ sont donnés par (3) pour le GP δ_j seul, sur ses résidus $\boldsymbol{d}_j$.

Preuve. Retirer $\boldsymbol{x}_i$ de tous les niveaux laisse le plan imbriqué. La proposition 1 donne alors $\mu_{j,-i} = \rho_{j-1}\mu_{j-1,-i} + \mu_{-i}^{\delta_j}$. Comme $z_j(\boldsymbol{x}_i) = \rho_{j-1} z_{j-1}(\boldsymbol{x}_i) + d_j(\boldsymbol{x}_i)$, on soustrait ; on raisonne de même sur les variances, grâce à l'indépendance des δ_j. □

Le terme de l'éq. 8 de [1], $(e_j - \hat{\rho}\, e_{j-1})^2/(s_j^2 - \hat{\rho}^2 s_{j-1}^2)$, est donc exactement le z_i^2 du GP `gps[j-1]`. Il se calcule à partir de son facteur de Cholesky, sans réajustement ni éq. 6–7. Chez Le Gratiet, $\hat{\rho}$ est ré-estimé sans le point i (moindres carrés généralisés) ; chez nous, ρ est appris avec le noyau et reste fixé, comme les autres hyperparamètres. Vérification par `CoKriging.leave_one_out(level, by_location=True)` : écart de $1{,}6 \cdot 10^{-4}$ à 3 niveaux et de $1{,}2 \cdot 10^{-2}$ à 2 niveaux, en relatif à l'écart-type des erreurs (bruit BF relativement plus grand).

Variance ajustée et choix du point ([1], éq. 8–9) :

$$k_{\text{adj}}(\boldsymbol{x}) = \sum_{j=1}^{L} P_j^2\, \sigma_j^2(\boldsymbol{x})\, a_j(\boldsymbol{x}), \qquad a_j(\boldsymbol{x}) = 1 + z^2_{j,\, i_j(\boldsymbol{x})}, \qquad \boldsymbol{x}_0 = \arg\max_{\boldsymbol{x} \in \mathcal{Q}} k_{\text{adj}}(\boldsymbol{x}). \tag{4}$$

$i_j(\boldsymbol{x})$ est le point de $\mathcal{D}_j$ dont la cellule de Voronoï contient $\boldsymbol{x}$, c'est-à-dire son plus proche voisin. [1] prennent la distance euclidienne dans les unités brutes, qui dépend du choix des unités (degrés de gîte contre nœuds de vitesse). **Nous proposons la métrique du noyau**, $\sum_m (u_m - u_{i,m})^2/\theta_{j,m}^2$: pour un noyau stationnaire, le plus proche voisin est alors le point *le plus corrélé* à $\boldsymbol{x}$. Ce choix est invariant par changement d'unités et suit l'anisotropie.

Subtilité — ce que fait vraiment l'ajustement

- Si le modèle est bien calibré, $\mathbb{E}[z^2] = 1$: $a_j \approx 2$ partout, et arg max ne bouge pas. L'ajustement n'agit que là où les erreurs observées s'écartent de la variance prédite.
- Il ne fait qu'*augmenter* la variance, jamais la réduire là où le modèle est trop prudent.
- k_{adj} est discontinu aux frontières des cellules. L'évolution différentielle de SciPy (déjà utilisée par `EGOOptimizer._find_next_point`) le supporte ; il n'est pas besoin de construire la tessellation.
- Avec peu de points, z^2 est bruité : un seul calcul aberrant gonfle toute sa cellule. Borner $z^2 \le z^2_{\max}$ (par exemple 9) est une option prudente, qui n'est pas dans [1].
- Le maximum de variance tend à tomber sur les bords du domaine (comportement « ALM », voir [11, ch. 6]). C'est attendu ; la variante du § 7 corrige ce biais.

4 Jusqu'à quel niveau : réduction d'IMSE par unité de coût

L'objectif global est l'erreur quadratique intégrée $\text{IMSE}_L = \int_{\mathcal{Q}} \text{Var}[Z_L(\boldsymbol{x}) \mid z]\, d\boldsymbol{x}$ [7]. Calculer $\boldsymbol{x}_0$ aux niveaux $1..l$ revient à observer $d_j(\boldsymbol{x}_0) = \delta_j(\boldsymbol{x}_0) + \varepsilon_j$ pour $j \le l$. Chaque δ_j concerné subit une mise à jour de rang un, et les autres ne changent pas :

$$\sigma_j^2(\boldsymbol{x}) \;\longrightarrow\; \sigma_j^2(\boldsymbol{x}) - \frac{k_j^n(\boldsymbol{x}, \boldsymbol{x}_0)^2}{\sigma_j^2(\boldsymbol{x}_0) + \tau_j^2}.$$

Avec (2), le gain sur la sortie haute fidélité est donc *exactement*, à hyperparamètres fixés,

$$G_l(\boldsymbol{x}_0) = \sum_{j=1}^{l} P_j^2\, a_j(\boldsymbol{x}_0) \int_{\mathcal{Q}} \frac{k_j^n(\boldsymbol{x}, \boldsymbol{x}_0)^2}{\sigma_j^2(\boldsymbol{x}_0) + \tau_j^2}\, d\boldsymbol{x} \;\approx\; \sum_{j=1}^{l} P_j^2\, a_j(\boldsymbol{x}_0)\, \frac{1}{N} \sum_{k=1}^{N} \frac{k_j^n(\boldsymbol{u}_k, \boldsymbol{u}_0)^2}{\sigma_j^2(\boldsymbol{u}_0) + \tau_j^2}, \tag{5}$$

avec N points de Sobol $\boldsymbol{u}_k$ dans le cube unité. Le volume $|\mathcal{Q}|$, commun à tous les niveaux, ne change pas le choix. Le facteur $a_j(\boldsymbol{x}_0)$ reprend l'éq. 10 de [1] (variance ajustée au point ajouté). Le gain ne dépend pas de la valeur que le code va rendre : il est connu *avant* de lancer le calcul. C'est le principe des stratégies de réduction pas à pas de l'incertitude (SUR) [8]. Le niveau est choisi au **meilleur rendement**, en incertitude retirée par unité de temps de calcul, comme dans le critère MR-SUR multi-fidélité de [9] :

$$l^\star = \arg\max_{l=1..L} \frac{G_l(\boldsymbol{x}_0)}{C_l}, \qquad C_l = \sum_{j=1}^{l} c_j \quad \text{(coût cumulé : l'imbrication impose les niveaux inférieurs).} \tag{6}$$

Calculer (5) coûte $O(Nn_j)$ par niveau, et seulement au point $\boldsymbol{x}_0$ retenu : c'est négligeable. Vérification contre un reconditionnement complet (`CoKriging.condition` avec $\boldsymbol{x}_0$ ajouté aux niveaux $1..l$) : écart relatif de 10^{-5} à $2 \cdot 10^{-3}$.

À retenir — pourquoi corriger la règle publiée

Pour $L = 2$, [1] approchent la réduction par $\sigma_j^2(\boldsymbol{x}_0)\, V_j$ avec un « volume d'influence » $V_j = \prod_m \theta_{j,m}$ (éq. 5). On note $R_1 = \sigma_1^2 V_1$ et $B = c_2/c_1$.

— **Unités.** $\mathrm{IMSE}^1_{\mathrm{red}} = R_1$ est une réduction de l'IMSE de la sortie *BF*, alors que $\mathrm{IMSE}^2_{\mathrm{red}} = \rho^2 R_1 + \sigma_2^2 V_2$ concerne la sortie *HF*. Or un calcul BF réduit l'IMSE de la HF de $\rho^2 R_1$. Comparons les deux règles, avec le même proxy :

$$[1] : B\, R_1 > \rho^2 R_1 + \sigma_2^2 V_2 \iff (B - \rho^2)\, R_1 > \sigma_2^2 V_2; \qquad \text{éq. (6)} : B\, \rho^2 R_1 > \sigma_2^2 V_2.$$

Les deux ne coïncident que pour $\rho^2 \approx 1$. Si $\rho^2 \geq B$, la règle publiée ne lance *jamais* la BF ; si $\rho < 1$, elle la lance trop. Dans mfego, $\hat{\rho} \approx 2{,}4$ sur les données de l'EGO (Hartmann), et la v3 monte jusqu'à 25.

— **Volume.** V_j n'est pas borné : sur notre cas test, $\theta_2 = (3{,}4;\ 4{,}8)$ donne $V_2 = 16$, pour un domaine de volume 1. L'intégrale de (5) est bornée par construction.

Cas test (2D, 2 niveaux, plan 40/15, $\hat{\rho} = 1{,}99$, 256 points candidats). Le résidu δ_2 est lisse et déjà bien appris : 99 % de la variance HF vient du niveau 1, la bonne réponse est donc la BF. Part des points où la BF est choisie :

	règle de [1] (algorithme 1 complet)	proxy corrigé	gain exact (5)–(6)
$B = 5$	0,85	0,97	1,00
$B = 20$	0,99	1,00	1,00

Le premier test de l'algorithme 1, $\sigma_l^2(\boldsymbol{x}_0) < \overline{\sigma_l^2}$, devient inutile : (6) compare directement les gains.

5 Algorithme et intégration dans le code

Algorithme 1 Co-krigeage séquentiel (une évaluation par itération)

Entrées : `data` (`ExperimentData` imbriqué et évalué), `simulator` (`BaseSimulator`), `data.costs`, budget, N, seuil Q^2_{cible}
tant que budget restant $\geq c_1$ **et** $Q^2_{\text{LOO}} < Q^2_{\text{cible}}$ **faire**
  `model = CoKriging(L)` ; `model.fit(data)` ▷ hyperparamètres ré-estimés
  $z^2_{j,i} \leftarrow$ (3) sur chaque `model.gps[j-1]` ▷ proposition 2
  $\boldsymbol{x}_0 \leftarrow \arg\max k_{\text{adj}}$ (4) par évolution différentielle, masque `feasibility`
  $l^\star \leftarrow \arg\max_l G_l(\boldsymbol{x}_0)/C_l$ (5)–(6) avec N points de Sobol
  pour $j = 1, \ldots, l^\star$ **faire** ▷ du moins cher au plus cher
    `y, metrics = simulator.evaluate(x0, j)` ; `data.add_observation(j, x0, y, metrics)`
    si y non fini **alors sortir de la boucle pour** ▷ niveaux $> j$ non lancés : plan imbriqué
    fin si
  fin pour
  $Q^2_{\text{LOO}} \leftarrow$ `prediction_scores(*model.leave_one_out())["q2"]`
fin tant que
retourner `model`, `data`, historique (coût, Q^2_{LOO}, niveaux choisis)

Noyau de calcul. Les fonctions ci-dessous ont été testées contre le vrai `CoKriging` (écarts du § 6). Elles n'utilisent que des objets existants de mfego.

```python
import numpy as np
from scipy.linalg import cho_solve, solve_triangular

def level_weights(model):                    # P_j, j = 1..L (single access to a private helper)
    return model._factors(np.array([model.num_levels]))[0]

def posterior_cov(gp, u_a, u_b):             # k_j^n(u_a, u_b), raw units, latent (no noise)
```

```
    v_a = solve_triangular(gp.l_chol, gp.kernel.get_cross_covariance_matrix(u_a, gp.x_train).T, lower=True)
    v_b = solve_triangular(gp.l_chol, gp.kernel.get_cross_covariance_matrix(u_b, gp.x_train).T, lower=True)
    return gp.y_std ** 2 * (gp.kernel.get_cross_covariance_matrix(u_a, u_b) - v_a.T @ v_b)

def level_variances(model, u):                # sigma_j^2(u), shape (L, m)
    return np.array([np.maximum(gp.predict_batch(u)[1] - gp.get_noise_variance(), 0.0) for gp in model.gps])

def loo_z2(gp):                               # Eq. (3): alpha_i^2 / [K^-1]_ii
    return gp.alpha ** 2 / np.diag(cho_solve((gp.l_chol, True), np.eye(len(gp.x_train))))

def adjustment(model, z2, u):                 # a_j(u) = 1 + z^2 of the most correlated point of D_j
    out = []
    for j, gp in enumerate(model.gps):
        d2 = np.sum(((u[:, None, :] - gp.x_train[None, :, :]) / gp.kernel.lengthscale) ** 2, axis=2)
        out.append(1.0 + z2[j][np.argmin(d2, axis=1)])
    return np.array(out)

def adjusted_variance(model, z2, u):          # Eq. (4)
    p2 = level_weights(model)[:, None] ** 2
    return np.sum(p2 * level_variances(model, u) * adjustment(model, z2, u), axis=0)

def delta_imse(model, z2, u0, u_int):         # Eq. (5): G_l(u0) for l = 1..L, u0 of shape (1, d)
    p2, a, gains = level_weights(model) ** 2, adjustment(model, z2, u0)[:, 0], []
    for j, gp in enumerate(model.gps):
        k_x0, s2_0 = posterior_cov(gp, u_int, u0)[:, 0], posterior_cov(gp, u0, u0)[0, 0]
        gains.append(p2[j] * a[j] * np.mean(k_x0 ** 2) / (s2_0 + gp.get_noise_variance()))
    return np.cumsum(gains)

def choose_level(model, z2, u0, u_int, costs):  # Eq. (6)
    return int(np.argmax(delta_imse(model, z2, u0, u_int) / np.cumsum(costs))) + 1
```

Les points de Sobol se tirent par `u_int = scipy.stats.qmc.Sobol(d, seed=s).random(N)`, déjà dans le cube unité ; $N \approx 1024$ à 4096 suffit pour $d \lesssim 10$. La recherche de $\boldsymbol{x}_0$ réutilise l'enveloppe vectorisée de `EGOOptimizer._find_next_point`, qui minimise `-adjusted_variance(model, z2, model._unit(x))`.

Dans le code — intégration dans `vpp_acceleration`

- **Fichiers.** Le module va dans `vpp_acceleration/src/sequential_design.py`, les scripts dans `vpp_acceleration/`, les tests dans `vpp_acceleration/tests/`. mfego n'est pas modifié.
- **Imports.** `mfego/src` n'a pas de `__init__.py` : c'est un *paquet espace de noms*. Si `vpp_acceleration/src` n'en a pas non plus, Python fusionne les deux dossiers sous le nom `src` (testé) : `sys.path.insert(0, ROOT/"mfego")` et `sys.path.insert(1, ROOT/"vpp_acceleration")`, puis `from src.cokriging import CoKriging` et `from src.sequential_design import ...`. Deux règles en découlent :
  - **aucun** `__init__.py` dans `vpp_acceleration/src`, sinon ce dossier masque tout mfego ;
  - **aucun nom de module déjà pris** par mfego (`acquisition`, `cokriging`, `data_management`, `kernels`, `optimizer`, `run_utils`, `simulator`, `surrogate_models`, `validation`, `visualization`). Un doublon est ignoré sans message d'erreur (testé).
- **GP par niveau.** Seul `fit()` conditionne `model.gps`. `from_dict` et `from_multifidelity` ne conditionnent que le système joint. Pour un modèle rechargé, il faut reconditionner : `gp.condition(model._unit(X_j), d_j)`, avec `d_j = y_j - rho * y_prev[nested_index(X_j, X_prev)]`.
- **Méthodes privées.** `_factors` et `_unit` sont privées : n'y accéder que via `level_weights` et une fonction `to_unit`, pour n'avoir qu'un endroit à corriger si mfego change.
- **Échecs.** `add_observation` stocke un calcul raté en NaN, que `get_training_data` exclut. Comme les niveaux sont évalués dans l'ordre croissant et qu'on s'arrête au premier échec, `missing_points_for_nesting(data, L)` reste vide et `fit` ne lève pas d'erreur.
- **Coûts.** `data.costs` doit être dans une unité homogène (secondes CPU). Le coût est cumulé dans (6).

6 Validation

Tests unitaires (`pytest`). Entre parenthèses, l'écart mesuré sur le cas 2D, puis la tolérance proposée.

1. `adjusted_variance` avec $z^2 = 0$ égale la variance de `predict_batch(include_noise=False)` ($1{,}2 \cdot 10^{-4}$; 10^{-3}).
2. Proposition 2 contre `leave_one_out(level, by_location=True)` ($1{,}6 \cdot 10^{-4}$ à $1{,}2 \cdot 10^{-2}$; $5 \cdot 10^{-2}$).
3. `delta_imse` contre la baisse de variance moyenne après `condition()` avec $\boldsymbol{x}_0$ ajouté aux niveaux $1..l$ et une valeur arbitraire ($\leq 2 \cdot 10^{-3}$; 10^{-2}).
4. Après chaque itération, `missing_points_for_nesting` est vide.

Benchmark.

- *Problèmes :* Forrester (dossier `example/cokriging`), Hartmann (classe `HartmannMF` de `benchmarks/bench_lib.py`) et profil (classe `AirfoilMF` du dossier `benchmarks/cokriging`).

— *Référence :* plan imbriqué LHS (`generate_nested_design`) **à coût égal**, 10 graines.

— *Métriques :* `validation.prediction_scores` sur un jeu de test (RMSE relative, Q^2, couverture à 95 %, NLPD), et part de BF.

— *Variantes :* variance seule ou ajustée, règle de [1] ou règle (6).

— *Critère de succès :* RMSE inférieure à celle du LHS imbriqué, sans dégrader la couverture.

7 Limites et extensions

— ρ **ponctuel.** [1, 4] intègrent ρ (prior de Jeffreys). Sa variance a posteriori s'ajoute alors à (2). Chez nous, elle manque : la variance est optimiste avec peu de points HF, et le choix du niveau est sensible à $\hat{\rho}$. Correction possible, au premier ordre (krigeage universel, [6, éq. 2.42]) : ajouter $P_j^2\, r_j(\boldsymbol{x})^2/(\boldsymbol{f}^\top \widetilde{\boldsymbol{K}}_j^{-1}\boldsymbol{f})$, avec $\boldsymbol{f}$ le niveau $j-1$ centré sur $\mathcal{D}_j$ et $r_j(\boldsymbol{x}) = \tilde{\mu}_{j-1}(\boldsymbol{x}) - \boldsymbol{f}^\top \widetilde{\boldsymbol{K}}_j^{-1}\boldsymbol{k}_j(\boldsymbol{x})$. Ce terme reste à valider.

— **Bruit des niveaux inférieurs.** Les propositions 1–2 ne sont alors qu'approchées, à l'ordre $\rho^2\tau_{j-1}^2/\sigma_j^2$: à surveiller avec des solveurs mal convergés.

— **Variante ALC** ([11], ch. 6). Choisir $\boldsymbol{x}_0 = \arg\max_{\boldsymbol{x}} \max_l G_l(\boldsymbol{x})/C_l$ au lieu du maximum de variance. C'est plus coûteux, car il faut (5) à chaque candidat, mais cela évite le biais vers les bords et vise directement l'IMSE.

— **Calculs en parallèle.** La variance ne dépend pas des valeurs : une sélection gloutonne est exacte. On ajoute le point choisi avec une valeur fictive par `condition()`, puis on recommence. Cela remplace l'échantillonnage MCMC suivi de k-means de [1], § 3.3 ; pour les heuristiques de type « liar », voir [12].

— **Sorties multiples (VPP).** Il faut un `CoKriging` par coefficient o, avec une tolérance physique ε_o pour chacun. On agrège alors $\sum_o k_{\mathrm{adj},o}/\varepsilon_o^2$ pour le point et $\sum_o G_{l,o}/\varepsilon_o^2$ pour le niveau, et on s'arrête quand chaque RMSE LOO passe sous ε_o.

— **Région d'intérêt.** Tirer les points d'intégration de (5) parmi les états de fonctionnement de la VPP donne un IMSE pondéré [10], sans autre changement de code. Le critère d'arrêt peut alors être la RMSE LOO pondérée de [14].

Références

[1] L. Le Gratiet, C. Cannamela, *Cokriging-based sequential design strategies using fast cross-validation techniques for multi-fidelity computer codes*, Technometrics 57(3) :418–427, 2015.

[2] M. C. Kennedy, A. O'Hagan, *Predicting the output from a complex computer code when fast approximations are available*, Biometrika 87(1) :1–13, 2000.

[3] L. Le Gratiet, *Multi-fidelity Gaussian process regression for computer experiments*, thèse, Université Paris Diderot, 2013.

[4] L. Le Gratiet, J. Garnier, *Recursive co-kriging model for design of computer experiments with multiple levels of fidelity*, Int. J. Uncertainty Quantification 4(5) :365–386, 2014.

[5] O. Dubrule, *Cross validation of kriging in a unique neighborhood*, Mathematical Geology 15(6) :687–699, 1983.

[6] C. E. Rasmussen, C. K. I. Williams, *Gaussian Processes for Machine Learning*, MIT Press, 2006 (§ 2.7 et § 5.4.2).

[7] J. Sacks, W. J. Welch, T. J. Mitchell, H. P. Wynn, *Design and analysis of computer experiments*, Statistical Science 4(4) :409–423, 1989.

[8] J. Bect, D. Ginsbourger, L. Li, V. Picheny, E. Vazquez, *Sequential design of computer experiments for the estimation of a probability of failure*, Statistics and Computing 22(3) :773–793, 2012.

[9] R. Stroh, J. Bect, S. Demeyer, N. Fischer, D. Marquis, E. Vazquez, *Sequential design of multi-fidelity computer experiments : maximizing the rate of stepwise uncertainty reduction*, Technometrics 64(2) :199–209, 2022.

[10] V. Picheny, D. Ginsbourger, O. Roustant, R. T. Haftka, N.-H. Kim, *Adaptive designs of experiments for accurate approximation of a target region*, J. Mechanical Design 132(7) :071008, 2010.

[11] R. B. Gramacy, *Surrogates : Gaussian Process Modeling, Design and Optimization for the Applied Sciences*, Chapman & Hall/CRC, 2020 (ch. 6, ALM et ALC).

[12] D. Ginsbourger, R. Le Riche, L. Carraro, *Kriging is well-suited to parallelize optimization*, in *Computational Intelligence in Expensive Optimization Problems*, Springer, 2010.

[13] J. P. C. Kleijnen, W. C. M. van Beers, *Application-driven sequential designs for simulation experiments : kriging metamodelling*, J. Operational Research Society 55 :876–883, 2004.

[14] J. Westermann, A. Zea, U. D. Hanebeck, *Adaptive sampling for global meta modeling using a Gaussian process variance measure*, European Control Conference, 573–579, 2021.